

Notas de Estudio - Período 3

Machine Learning Básico con Scikit-Learn

1 Introducción al concepto de modelo

Cuando hablamos de un «modelo» en este curso, es útil pensar en la analogía del fútbol. Un modelo no es simplemente una estructura estática que observa datos, sino más bien como un entrenador que aprende patrones a partir de la experiencia. Este entrenador estudia partidos anteriores, analiza qué estrategias funcionaron y cuáles no, identifica relaciones entre diferentes factores del juego, y eventualmente desarrolla una intuición sobre cómo predecir resultados futuros. La clave está en que este proceso es sistemático y basado en datos, no en meras suposiciones.

1.1 El proceso de entrenamiento

Entrenar un modelo significa fundamentalmente ajustar sus parámetros internos utilizando datos históricos. Es importante entender qué NO es entrenar un modelo: no se trata de memorizar únicamente los resultados exactos de cada caso, no consiste en editar manualmente las respuestas que queremos obtener, y definitivamente no implica mezclar datos de prueba y entrenamiento sin ningún orden metodológico.

El entrenamiento es un proceso iterativo y elegante. El modelo examina ejemplos del pasado, identifica patrones subyacentes en esos datos, y gradualmente ajusta sus parámetros internos para capturar esos patrones. Lo que buscamos es que el modelo aprenda a generalizar, es decir, que pueda hacer predicciones razonables sobre casos que nunca ha visto antes, no que simplemente recite de memoria lo que ocurrió en el pasado.

2 La división fundamental: entrenamiento y prueba

Uno de los conceptos más importantes en machine learning es la separación rigurosa entre datos de entrenamiento y datos de prueba. En scikit-learn, esta división se realiza mediante la función `train_test_split()`, que toma nuestros datos y los separa aleatoriamente en dos conjuntos. La sintaxis básica es elegante y directa:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Consideremos un ejemplo concreto para entender las proporciones. Si tenemos un dataset con información de 250 partidos de fútbol y especificamos `test_size=0.2`, estamos indicando que queremos reservar el 20% de los datos para prueba. Esto significa que aproximadamente 50 partidos se guardarán para evaluar el modelo, mientras que los 200 restantes (el 80%) se utilizarán para entrenarlo. Esta división es crucial porque nos permite evaluar honestamente qué tan bien funciona nuestro modelo en datos que no ha visto durante el entrenamiento.

3 El problema del sobreajuste

El overfitting, o sobreajuste, es uno de los desafíos más sutiles y peligrosos en machine learning. Volviendo a nuestra analogía futbolística, el overfitting es como un entrenador que ha memorizado cada jugada específica de los partidos anteriores sin realmente entender los principios fundamentales del juego. Este entrenador podría recitar perfectamente qué ocurrió en cada partido pasado, pero sería incapaz de adaptar sus estrategias a nuevas situaciones.

¿Cómo identificamos el overfitting en la práctica? El síntoma clásico es una discrepancia marcada entre el rendimiento en entrenamiento y en prueba. Si nuestro modelo muestra una precisión muy alta cuando lo evaluamos con los datos que usó para aprender, pero una precisión considerablemente más baja cuando lo probamos con datos nuevos, tenemos un caso claro de sobreajuste. El modelo memorizó los datos de entrenamiento en lugar de aprender patrones generalizables.

4 Modelos de clasificación: regresión logística y random forest

En este curso trabajamos principalmente con dos modelos de clasificación, cada uno con sus propias fortalezas y características. La regresión logística, a pesar de su nombre confuso, es en realidad un modelo para clasificación binaria. Es el modelo más simple que utilizamos, y esa simplicidad es precisamente su virtud. La regresión logística es altamente interpretable, lo que significa que podemos entender relativamente fácil cómo está tomando sus decisiones. Es un excelente punto de partida para cualquier problema de clasificación.

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression()  
lr.fit(X_train, y_train)  
y_pred = lr.predict(X_test)
```

Por otro lado, Random Forest es considerablemente más complejo. Se trata de un ensemble, es decir, un conjunto de múltiples árboles de decisión que trabajan juntos. Cada árbol en el bosque hace su propia predicción, y el resultado final se determina mediante una especie de votación. Random Forest tiende a ser más robusto que modelos más simples y puede capturar relaciones no lineales complejas en los datos. Sin embargo, esta complejidad adicional viene con un costo: es más difícil interpretar exactamente por qué el modelo está haciendo determinadas predicciones.

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier()  
rf.fit(X_train, y_train)  
y_pred = rf.predict(X_test)
```

5 Características: selección e importancia

Las características, o features, son las variables que alimentamos a nuestro modelo. La forma correcta de especificar características en Python es mediante una lista de strings con los nombres exactos de las columnas. Por ejemplo, si nuestro DataFrame contiene columnas como `goles_local`, `goles_visitante`, y `tiros_local`, especificaríamos nuestras características así:

```
features = ['goles_local', 'goles_visitante', 'tiros_local']  
X = df[features]
```

Es crucial usar comillas y asegurarnos de que los nombres coincidan exactamente con los del DataFrame. Un error común es olvidar las comillas o intentar pasar las variables directamente sin strings.

Ahora bien, ¿por qué no simplemente incluir todas las columnas disponibles? La respuesta es que más no siempre es mejor. Incluir demasiadas columnas irrelevantes puede introducir ruido en nuestro modelo y empeorar su capacidad de generalización. Variables que no tienen una relación real con lo que queremos predecir pueden confundir al modelo, hacer que el entrenamiento tome más tiempo, y reducir la interpretabilidad de los resultados.

Random Forest nos proporciona una herramienta útil para evaluar qué características son realmente importantes: la métrica de importancia de variables. Esta métrica nos indica la contribución de cada característica a la hora de reducir los errores de predicción. En términos prácticos, las características con mayor importancia son aquellas que aportan más información a las divisiones internas de los árboles del bosque. Cuando vemos que una variable tiene alta importancia, sabemos que está jugando un papel significativo en las predicciones del modelo.

```
importances = rf.feature_importances_  
# Esto nos da un array con la importancia de cada característica
```

6 Variables derivadas: creando nuevas características

Una de las técnicas más poderosas en machine learning es la creación de variables derivadas. Estas son nuevas características que construimos a partir de las existentes, con la esperanza de que capturen información relevante de una manera más directa. Un ejemplo simple pero efectivo es crear una variable `total_goles` que suma los goles del equipo local y visitante:

```
df['total_goles'] = df['goles_local'] + df['goles_visitante']
```

Otro ejemplo particularmente útil es la diferencia de goles, que puede ser muy predictiva del resultado del partido:

```
df['diferencia_goles'] = df['goles_local'] - df['goles_visitante']
```

¿Cómo sabemos si una variable derivada es útil? La respuesta está en experimentar y observar el impacto en el rendimiento del modelo. Si añadimos `diferencia_goles` como característica y vemos que la precisión de nuestras predicciones mejora, esto sugiere fuertemente que la diferencia de goles contiene información predictiva valiosa. Está capturando algo sobre el desempeño relativo de los equipos que es relevante para determinar el resultado.

Por el contrario, si eliminamos una variable y la precisión se mantiene igual, hemos aprendido que esa variable no estaba aportando información útil. En ese caso, es mejor mantener el modelo más simple eliminando esa característica redundante.

7 Métricas de evaluación: midiendo el éxito

La precisión, o accuracy, es la métrica más intuitiva para evaluar modelos de clasificación. Se define matemáticamente como:

$$\text{Precisión} = \frac{\text{Predicciones correctas}}{\text{Total de predicciones}}$$

Consideremos un ejemplo concreto. Si nuestro modelo hace predicciones sobre 50 partidos y acierta el resultado en 34 de ellos, la precisión sería $\frac{34}{50} = 0.68$, o 68%. En scikit-learn, calcular esta métrica es directo:

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
```

Sin embargo, la precisión por sí sola no siempre cuenta toda la historia. Aquí es donde entra la matriz de confusión, una herramienta que nos da una visión más detallada de los errores del modelo. La matriz de confusión desglosa nuestras predicciones en cuatro categorías: verdaderos positivos (casos positivos que predijimos correctamente), falsos positivos (casos negativos que erróneamente clasificamos como positivos), verdaderos negativos (casos negativos correctamente clasificados), y falsos negativos (casos positivos que no logramos identificar).

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
```

	Predicho: Positivo	Predicho: Negativo
Real: Positivo	Verdadero Positivo	Falso Negativo
Real: Negativo	Falso Positivo	Verdadero Negativo

Esta matriz nos permite ver no solo cuántas predicciones fueron correctas, sino también qué tipos de errores está cometiendo el modelo.

8 La importancia del baseline

Antes de celebrar los resultados de nuestro modelo, necesitamos un punto de referencia: el baseline. El baseline más simple es predecir siempre la clase mayoritaria. Por ejemplo, si en nuestro dataset de 40 partidos hay 18 victorias locales y 22 victorias visitantes, el baseline sería predecir siempre «victoria visitante», lo cual nos daría una precisión de $\frac{22}{40} = 0.55$ o 55%.

Ahora supongamos que construimos un modelo de machine learning y este obtiene una precisión de 0.57 en los datos de prueba. ¿Es esto bueno? Bueno, estamos apenas 2 puntos porcentuales por encima del baseline. Esta mejora marginal debe analizarse cuidadosamente. ¿Vale la pena la complejidad adicional del modelo para obtener solo una mejora del 2%? Tal vez en algunos contextos sí, pero en muchos casos podríamos concluir que el modelo no está aportando suficiente valor como para justificar su uso.

El propósito del baseline es precisamente este: nos mantiene honestos. Nos obliga a preguntarnos si nuestro modelo realmente está aprendiendo patrones útiles o si apenas está haciendo algo mejor que una estrategia trivial. Si nuestro modelo no puede superar consistentemente al baseline, probablemente necesitemos repensar nuestro enfoque.

9 Comparando modelos: simplicidad versus complejidad

Supongamos que entrenamos tanto una regresión logística como un Random Forest en el mismo problema. Random Forest obtiene una precisión de 0.70, mientras que la regresión logística obtiene 0.68. La diferencia es pequeña pero favorable al modelo más complejo. ¿Qué hacemos?

Una acción razonable es conservar ambos modelos inicialmente y documentar esta diferencia ligera. No deberíamos apresurarnos a declarar la regresión logística como inválida simplemente porque el otro modelo tiene una precisión marginalmente superior. De hecho, hay argumentos sólidos para preferir el modelo más simple cuando el rendimiento es similar.

La interpretabilidad es valiosa. Con regresión logística, podemos entender relativamente fácil cómo cada característica está influyendo en las predicciones. Con Random Forest, esta interpretación es mucho más opaca. Si vamos a implementar este modelo en un contexto donde necesitemos explicar las decisiones, el modelo más simple podría ser preferible a pesar de su ligera desventaja en precisión.

En este curso limitamos deliberadamente el número de modelos que exploramos. No estamos tratando de convertirnos en expertos en todas las técnicas de machine learning que existen. El objetivo es centrarnos en los fundamentos, desarrollar intuición sobre cómo funcionan estos algoritmos, y evitar la confusión que viene de intentar aprender demasiadas técnicas simultáneamente. Una vez que estos fundamentos están sólidos, será mucho más fácil expandir a técnicas más avanzadas.

10 Análisis experimental: interpretando cambios

Cuando hacemos cambios a nuestro modelo, necesitamos interpretar cuidadosamente los resultados. Supongamos que nuestro modelo inicial tiene una precisión de 0.62, y después de añadir la característica `diferencia_goles`, la precisión sube a 0.66. El incremento absoluto es simplemente $0.66 - 0.62 = 0.04$, o 4 puntos porcentuales.

Este experimento nos enseña algo importante: la diferencia de goles contiene información predictiva valiosa. El modelo está utilizando esta nueva característica para hacer mejores predicciones. No es sorprendente cuando lo pensamos, la diferencia de goles es un resumen muy compacto del desempeño relativo de los equipos.

Pero no todos los experimentos son exitosos. ¿Qué aprendemos cuando añadimos una variable y la precisión cae? Esto nos enseña que las variables irrelevantes pueden introducir ruido. El modelo puede estar tratando de encontrar patrones en datos que fundamentalmente son aleatorios respecto a lo que queremos predecir, y este ruido interfiere con su capacidad de identificar los patrones reales.

Por otro lado, si añadimos o eliminamos una variable y la precisión se mantiene exactamente igual, esto sugiere que la variable no estaba aportando información nueva. Tal vez la información que contiene ya está capturada por otras características, o simplemente no es relevante para la predicción. En este caso, la simplicidad favorece eliminar la variable redundante.

11 Reflexión final: el flujo de trabajo completo

Cuando trabajamos en un proyecto de machine learning, seguimos un flujo general que se repite en cada iteración. Comenzamos preparando nuestros datos, lo cual incluye cargar el dataset y posiblemente crear variables derivadas que creemos puedan ser útiles. Luego dividimos estos datos en conjuntos de entrenamiento y prueba, típicamente usando una proporción de 80/20 u 70/30.

El siguiente paso es entrenar modelos. La estrategia prudente es comenzar con modelos simples como la regresión logística, establecer un baseline de rendimiento, y luego experimentar con modelos más complejos como Random Forest para ver si la complejidad adicional se justifica.

La evaluación viene después: calculamos la precisión, revisamos la matriz de confusión para entender qué tipos de errores estamos cometiendo, y crucialmente, comparamos nuestros resultados con el baseline. Si estamos usando Random Forest, también analizamos la importancia de las variables para entender qué características están siendo más útiles.

Finalmente, iteramos. Tal vez probamos nuevas variables derivadas basándonos en lo que hemos aprendido sobre la importancia de las características. Quizás simplificamos el modelo eliminando variables que no aportan. A través de este proceso iterativo, refinamos gradualmente nuestro enfoque, siempre manteniendo en mente que la simplicidad es una virtud cuando el rendimiento es comparable.